



Department of Electrical Engineering
Linear Control Project Report

Python Final Project Linear Control Course

Amirmohammad Ahmadizad

Student Number: 401411057

Amirmasoud Derakhshan Hesar

Student Number: 401411498

Winter 2026

Interactive Step Response Analyzer for Linear Control Systems

Introduction

In linear time-invariant (LTI) system theory, the step response $y_s(t)$ of a system is its zero-state output when subjected to a unit step input $u(t)$.

Unit Step Input:

$$u(x) = \begin{cases} 0, & t < 0 \\ 1, & t \geq 0 \end{cases}$$

Mathematical Relation: The step response is defined as the convolution integral of the system's impulse response $h(t)$:

$$y_s(t) = \int_0^t h(\tau) d\tau$$

Equivalently, in the Laplace domain:

$$Y_s(s) = \frac{1}{s} H(s)$$

Where $H(s)$ is the system's transfer function.

The step response serves as a standard measure for analyzing and evaluating the transient performance of control systems. Its quantitative characteristics—rise time, overshoot, settling time, and steady-state error—are directly used in controller design specifications and are fundamentally linked to the pole locations of the system in the S-plane.

This project implements an advanced interactive tool for analyzing and comparing step responses of linear control systems. The software enables simultaneous visualization of both arbitrary transfer functions and canonical second-order systems, providing comprehensive metrics for performance evaluation. The tool serves as an educational and engineering platform for understanding system dynamics, stability characteristics, and performance parameters through real-time parameter manipulation.

Primary Project Objectives:

- Comparative System Analysis: Enable side-by-side comparison of arbitrary transfer functions and second-order systems.
- Performance Metrics Calculation: Automatically compute key performance indicators (overshoot, settling time, rise time).
- Parameter Sensitivity Analysis: Study effects of parameter variations on system response.

Core Architecture and Implementation

1. Mathematical Foundation

Transfer Function Representation:

The system implements continuous-time transfer functions in polynomial form:

$$G(s) = \frac{N(s)}{D(s)} = \frac{a_0 + a_1s + \dots}{b_0 + b_1s + \dots}$$

Second-Order Canonical Form:

Standard second-order system implementation:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_ns + \omega_n^2}$$

where:

- ζ (zeta): Damping ratio
- ω_n : Natural frequency

2. Module Structure

a) Data Processing and Validation Module:

```
def parse_poly_list(s: str):
```

Converts string input of polynomial coefficients to numerical list.

Processing Steps:

- String normalization (space to comma conversion)
- Empty string detection and validation
- Floating-point conversion with error handling
- List generation for control system functions

```
def safe_tf(num, den):
```

Validates and creates transfer function objects.

Key Validations:

- Denominator non-emptiness check
- Leading coefficient non-zero verification
- Proper scaling and normalization
- Compatibility with control library requirements

b) System Analysis and Characterization Module:

```
def dominant_complex_pole(poles):
```

Identifies the dominant complex pole for system characterization.

Algorithm:

- Filters real poles (imaginary part)
- Selects poles with largest real part (closest to instability boundary)
- Returns None for purely real systems

Significance: Dominant poles dictate transient response characteristics.

```
def zeta_wn_from_pole(p):
```

Extracts damping ratio and natural frequency from complex pole.

$$\omega_n = |p|, \zeta = \frac{\text{Re}(p)}{\omega_n}$$

Relates pole positions to physical system parameters.

c) Performance Metrics Computation Module:

```
def step_metrics(t, y, tol=0.02):
```

Computes quantitative performance measures from step response.

Metrics Calculated:

- Final Value (y_{final}): Steady-state response
- Percentage Overshoot (OS%):

$$OS\% = \left(\left(\frac{y_{\text{max}} - y_{\text{final}}}{y_{\text{final}}} \right) \times 100 \right) \%$$

- Settling Time (T_s): Time to enter and remain within $\pm \text{tol}\%$ band:
 - 2% tolerance default (industry standard)
 - Backward scanning algorithm for accuracy
- Rise Time (T_r): Time from 10% to 90% of final value:
 - Direction-sensitive detection for positive/negative responses
 - Linear interpolation between sample points

Algorithm Features:

- Robust handling of noisy data
- Edge case management (monotonic responses)
- Direction-aware threshold crossing

d. Second-Order System Generation Module:

```
def make_2nd_order_tf(zeta, wn):
```

Creates canonical second-order transfer function.

Implementation:

$$\text{Coefficient: } \frac{\omega_n^2}{1, 2\zeta\omega_n, \omega_n^2}$$

3. User Interface Architecture

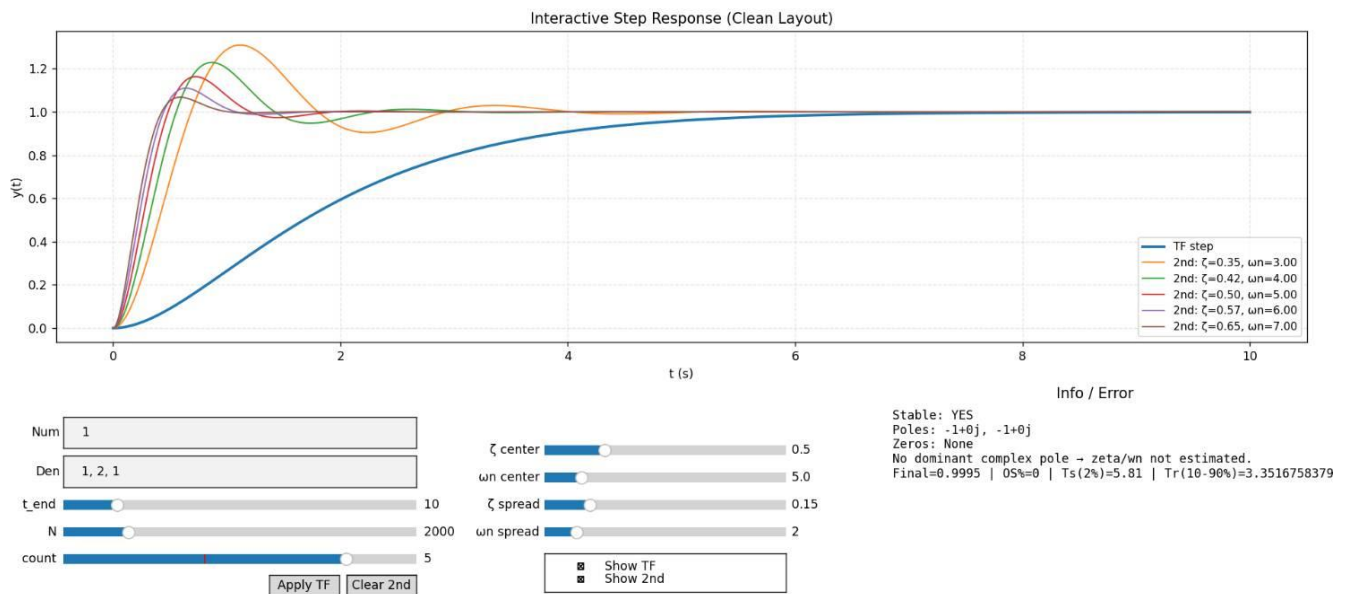
A. Layout Management System:

Grid Specification:

```
gs = fig.add_gridspec(
    nrows=2, ncols=2,
    height_ratios=[3.2, 2.0],
    width_ratios=[3.5, 2.0],
    hspace=0.25, wspace=0.15
)
```

Component Distribution:

- Top Panel (ax_plot): Main response plot spanning both columns
- Bottom-Left Panel: Control interface split into:
 - Transfer function controls (ax_ctrlL)
 - Second-order system controls (ax_ctrlR)
- Bottom-Right Panel (ax_info): Information display and error reporting



Conclusion

This project successfully implements a sophisticated interactive tool for control system analysis that bridges theoretical concepts with practical visualization. The software provides:

- Comprehensive Analysis: From arbitrary transfer functions to performance metrics
- Intuitive Interaction: Real-time parameter manipulation with immediate feedback
- Engineering Utility: Practical tools for system design and analysis

The implementation demonstrates excellent software engineering practices while maintaining focus on the educational and analytical goals of the project. The tool serves as both a learning aid for students and a practical utility for control system designers.